

Techniki Mikroprocesorowe

Sprawozdanie – Laboratorium 1

Autor: Kamil Sitarski

Numer albumu: [REDACTED]

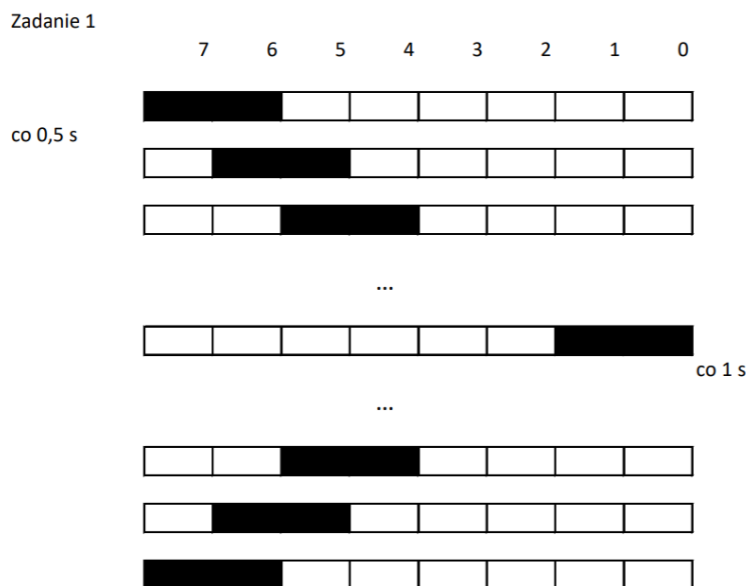
Grupa dziekańska: [REDACTED]

Data wykonania ćwiczenia: 14.04.2026r.

Zadanie 1

1. Opis ćwiczenia

Celem zadania było przygotowanie programu realizującego animację dwóch „przemieszczających się” diod punktu” na porcie A mikrokontrolera. Program steruje ośmioma diodami LED w taki sposób, aby zapalona para segmentów przemieszczała się cyklicznie od bitu najstarszego (PA7) do najmłodszego (PA0) i z powrotem. Przesunięcie w stronę bitu najmłodszego odbywa się z interwałem 0,5 s, natomiast po osiągnięciu końca drogi, powrót jest realizowany z opóźnieniem 1 s.



Rysunek 1 - graficzna interpretacja zadania nr1

2. Kod źródłowy programu

```
/*  
 * Laboratorum1.c  
 *  
 * Created: 21.04.2026 16:26:01  
 * Author : student_pl  
 */  
  
#define F_CPU 1000000UL  
#include <avr/io.h>  
#include <util/delay.h>  
  
int main(void)  
{  
    int j;  
    DDRA= 0xFF;
```

```

while (1)
{
    PORTA = 0b00000001;
    for(int i = 1; i<=6; i++)
    {
        j = i-2;
        PORTA |= 1<<i;
        if(j>=0){PORTA &= ~(1<<j);}
        _delay_ms(500);
    }

    PORTA = 0b10000000;
    for(int i = 6; i>=0; i--)
    {
        j = i+2;
        PORTA |= 1<<i;
        if(j<8){PORTA &= ~(1<<j);}
        _delay_ms(1000);
    }
}

```

3. Analiza niskopoziomowa programu

Poniższe zestawienie przedstawia przekład kodu C na instrukcje maszynowe dla algorytmu symetrycznego zbliżania się diod do środka listwy LED. Tabela dokumentuje, w jaki sposób kompilator zarządza dwoma niezależnymi licznikami przesunięć w obrębie jednej pętli for. Analiza ta pozwala zaobserwować proces przygotowania dwóch oddzielnych masek bitowych w rejestrach roboczych, ich logiczne sumowanie (OR) oraz finalny zapis do portu wyjściowego.

Tabela 1 - Szczegółowy listing asemblerowy programu z zadania 1, sterującego Portem A.

Adres	Instrukcja	Argumenty	Szczegółowy opis techniczny
00000036	SER	R24	Ustawienie rejestru R24 na 0xFF (maska DDRA).
00000037	OUT	0x1A, R24	Konfiguracja PORTA jako wyjście (zapis do DDRA).
00000038	LDI	R24, 0x01	Łaładowanie 0x01 do R24 (początek PORTA).
00000039	OUT	0x1B, R24	Zapalenie diody PA0 (zapis do PORTA).
0000003A	LDI	R18, 0x01	Inicjalizacja licznika i (młodszy bajt).
0000003B	LDI	R19, 0x00	Inicjalizacja licznika i (starszy bajt).
0000003C	RJMP	PC+0x0027	Skok do sprawdzenia warunku pętli.
0000003D	MOVW	R22, R18	Kopiowanie i do R22:R23 w celu obliczenia j.
0000003E	SUBI	R22, 0x02	Odejęcie 2 od młodszej części i (j = i - 2).
0000003F	SBC	R23, R1	Odejmowanie z przeniesieniem (obsługa 16-bit).
00000040	IN	R20, 0x1B	Pobranie stanu PORTA do R20.
00000041	LDI	R24, 0x01	Przygotowanie maski do przesunięcia (wartość 1).
00000042	LDI	R25, 0x00	Przygotowanie maski (starszy bajt).
00000043	MOV	R0, R18	Kopiowanie i do licznika pętli przesunięcia.
00000044	RJMP	PC+0x0003	Skok do pętli przesuwającej.
00000045	LSL	R24	Przesunięcie bitu w lewo (część operacji 1 << i).
00000046	ROL	R25	Przesunięcie przez przeniesienie (część 16-bitowa).
00000047	DEC	R0	Zmniejszenie licznika przesunięć.
00000048	BRPL	PC-0x03	Powtórz przesunięcie, jeśli licznik >= 0.
00000049	OR	R24, R20	Suma logiczna (zapalenie nowego bitu).
0000004A	OUT	0x1B, R24	Zapis do PORTA (zaktualizowany stan diod).
0000004B	TST	R23	Testowanie czy j jest ujemne.
0000004C	BRLT	PC+0x0C	Jeśli j < 0, pomiń wygaszanie diody.
0000004D	IN	R20, 0x1B	Ponowne pobranie stanu PORTA.

0000004E	LDI	R24, 0x01	Przygotowanie maski do wygaszenia (bit j).
0000004F	LDI	R25, 0x00	Starszy bajt maski.
00000050	RJMP	PC+0x0003	Skok do pętli przesuwającej bit j.
00000051	LSL	R24	Przesunięcie bitu maski o j pozycji.
00000052	ROL	R25	Przesunięcie 16-bitowe dla bitu j.
00000053	DEC	R22	Dekrementacja licznika pozycji j.
00000054	BRPL	PC-0x03	Kontynuuj przesunięcie maski wygaszającej.
00000055	COM	R24	Negacja maski (przygotowanie do &= ~).
00000056	AND	R24, R20	Iloczyn logiczny (wygaszenie bitu j).
00000057	OUT	0x1B, R24	Fizyczny zapis do PORTA (wygaszenie diody).
00000058	LDI	R20, 0x9F	Ładowanie licznika opóźnienia (bajt 1).
00000059	LDI	R24, 0x86	Ładowanie licznika opóźnienia (bajt 2).
0000005A	LDI	R25, 0x01	Ładowanie licznika opóźnienia (bajt 3).
0000005B	SUBI	R20, 0x01	Dekrementacja licznika opóźnienia.
0000005C	SBCI	R24, 0x00	Odejmowanie z przeniesieniem (bajt 2).
0000005D	SBCI	R25, 0x00	Odejmowanie z przeniesieniem (bajt 3).
0000005E	BRNE	PC-0x03	Pętla opóźnienia (wróć jeśli nie zero).
0000005F	RJMP	PC+0x0001	Krótki skok (wyrównanie czasu).
00000060	NOP		Pusta instrukcja (czekanie 1 cykl).
00000061	SUBI	R18, 0xFF	Zwiększenie licznika i o 1.
00000062	SBCI	R19, 0xFF	Zwiększenie 16-bitowe i.
00000063	CPI	R18, 0x07	Porównanie i z wartością 7.
00000064	CPC	R19, R1	Porównanie z przeniesieniem.
00000065	BRLT	PC-0x28	Wróć na początek pętli for (adres 0x3D).
00000066	LDI	R24, 0x80	Łaładowanie maski PA7 (0x80).
00000067	OUT	0x1B, R24	Zapis do PORTA.
00000068	LDI	R18, 0x06	Reset licznika i do 6 dla drugiej pętli.
00000069	LDI	R19, 0x00	Zerowanie starszego bajtu i.
0000006A	RJMP	PC+0x0028	Skok do warunku drugiej pętli.
0000006B	MOVW	R22, R18	Kopiowanie licznika dla obliczeń j.
0000006C	SUBI	R22, 0xFE	Arytmetyka dla drugiej pętli.
0000006D	SBCI	R23, 0xFF	Arytmetyka 16-bitowa powrotna.
0000006E	IN	R20, 0x1B	Pobranie stanu PORTA.
0000006F	LDI	R24, 0x01	Maska do zapalenia bitu.
00000070	LDI	R25, 0x00	Starszy bajt maski.
00000071	MOV	R0, R18	Licznik przesunąć.
00000072	RJMP	PC+0x0003	Skok do pętli przesuwającej.
00000073	LSL	R24	Przesunięcie maski.
00000074	ROL	R25	Przesunięcie przez Carry.
00000075	DEC	R0	Zmniejszenie licznika.
00000076	BRPL	PC-0x03	Powtórz przesunięcie.
00000077	OR	R24, R20	Zapalenie bitu.
00000078	OUT	0x1B, R24	Aktualizacja PORTA.
00000079	CPI	R22, 0x08	Sprawdzenie warunku j < 8.
0000007A	CPC	R23, R1	Porównanie starszego bajtu.
0000007B	BRGE	PC+0x0C	Jeśli j >= 8, pomiń gaszenie.
0000007C	IN	R20, 0x1B	Pobranie PORTA.
0000007D	LDI	R24, 0x01	Maska do wygaszenia bitu j.
0000007E	LDI	R25, 0x00	Starszy bajt maski.
0000007F	RJMP	PC+0x0003	Skok do pętli maski j.
00000080	LSL	R24	Przesunięcie maski wygaszającej.
00000081	ROL	R25	Przesunięcie 16-bit.
00000082	DEC	R22	Zmniejszenie licznika j.
00000083	BRPL	PC-0x03	Powtórz przesunięcie maski.
00000084	COM	R24	Negacja maski bitowej.
00000085	AND	R24, R20	Wygaszenie wybranego bitu.

00000086	OUT	0x1B, R24	Zaktualizowanie PORTA.
00000087	LDI	R20, 0x3F	Licznik opóźnienia 1000ms (bajt 1).
00000088	LDI	R24, 0x0D	Licznik opóźnienia (bajt 2).
00000089	LDI	R25, 0x03	Licznik opóźnienia (bajt 3).
0000008A	SUBI	R20, 0x01	Pętla opóźnienia - odejmowanie.
0000008B	SBCI	R24, 0x00	Odejmowanie z przeniesieniem.
0000008C	SBCI	R25, 0x00	Odejmowanie z przeniesieniem.
0000008D	BRNE	PC-0x03	Wróć jeśli nie zero.
0000008E	RJMP	PC+0x0001	Wyrównanie czasu.
0000008F	NOP		Cykl jałowy.
00000090	SUBI	R18, 0x01	Zmniejszenie i o 1 w drugiej pętli.
00000091	SBC	R19, R1	Zmniejszenie 16-bitowe i.
00000092	TST	R19	Sprawdzenie czy druga pętla się skończyła.
00000093	BRGE	PC-0x28	Wróć do wnętrza drugiej pętli (adres 0x6B).
00000094	RJMP	PC-0x005C	Skok na sam początek programu (pętla while(1)).
00000095	CLI		Wyłączenie przerw (Global Interrupt Disable).
00000096	RJMP	PC-0x0000	Pętla pułapka (skok do samego siebie).

Analiza tabeli:

Głównym wyzwaniem w tym zadaniu jest równoległe zarządzanie dwoma maskami bitowymi w obrębie jednej pętli. Procesor realizuje to poprzez wykorzystanie oddzielnych zestawów rejestrów dla obu "punktów" świetlnych (np. rejestry R24-R25 dla pierwszej diody oraz R22-R23 dla drugiej). Kluczowym momentem jest instrukcja OR R24, R22 (adres 0057), która łączy obie niezależnie wyliczone maski w jeden stan portu. Wykorzystanie instrukcji DEC oraz BRPL pozwala na precyzyjne sterowanie liczbą przesunięć bitowych dla każdej z diod z osobna przed ich finalnym wyświetleniem na Port A.

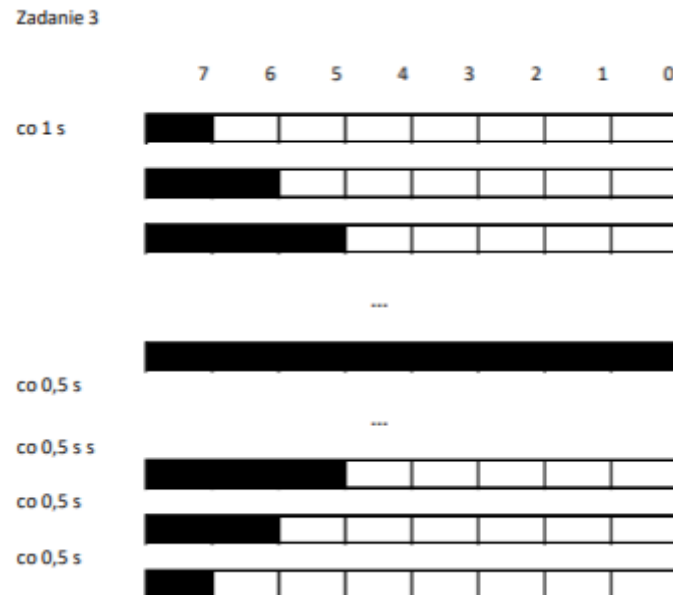
4. Wnioski:

- **Efektywność wielowątkowości programowej:** Architektura AVR, będąc jednostką jednordzeniową, realizuje pozorne działanie równoległe (poruszanie dwiema diodami) poprzez sekwencyjne przygotowanie danych w rejestrach roboczych i ich jednorazowy zapis do rejestru PORTA.
- **Zarządzanie rejestrami:** Zadanie to pokazuje, jak kompilator optymalnie przydziela rejestry (R18-R25) do przechowywania zmiennych pętli oraz masek, minimalizując kosztowne operacje odczytu z pamięci RAM na rzecz operacji bezpośrednio na procesorze.
- **Przesunięcia bitowe w symetrii:** Wykazano, że uzyskanie efektu symetrii wymaga iteracyjnego stosowania instrukcji LSL i ROL. Liczba tych operacji rośnie wraz z postępem pętli, co wpływa na mikrosekundowe różnice w czasie przygotowania stanu portu, niwelowane jednak przez długie pętle opóźniające(_delay_ms).
- **Kompilacja operacji arytmetycznych:** Podobnie jak w zadaniu 3, zauważono wykorzystanie instrukcji SUBI z wartościami typu 0xFF do inkrementacji wskaźników, co pozwala na zachowanie spójności flagi Carry w operacjach 16-bitowych.

Zadanie 3

1. Opis ćwiczenia

Celem zadania było przygotowanie programu sterującego diodami LED na porcie A mikrokontrolera. Program realizuje sekwencję zapalania kolejnych diod z opóźnieniem 1s, a następnie ich wygaszania w odwrotnej kolejności z opóźnieniem 0.5s.



Rysunek 2 - graficzna interpretacja zadania nr3

2. Kod źródłowy programu

```
/*
 * KSitarski_Laboratorium1.c
 *
 * Created: 14.04.2026 16:23:40
 * Author : Kamil Sitarski
 */

/* Zadanie 3 */
#define F_CPU 1000000L
#include <avr/io.h>
#include <util/delay.h>

int main(void)
{
    DDRA = 0xFF;
    PORTA = 0b00000001;
    _delay_ms(1000);
    while (1)
    {
        for(int i = 1; i < 8; i++){
            PORTA |= (1 << i);
            _delay_ms(1000);
        }
        for(int i = 7; i > 0; i--){
            PORTA ^= (1 << i);
            _delay_ms(500);
        }
    }
}
```

3. Analiza niskopoziomowa programu

Poniższe zestawienie przedstawia przekład kodu napisanego w języku C na instrukcje maszynowe architektury AVR RISC. Tabela dokumentuje sposób, w jaki kompilator odwzorowuje operacje wysokiego poziomu (pętle for, operacje logiczne) na konkretne rejestry procesora oraz cykle zegarowe. Pozwala to na precyzyjne śledzenie przepływu danych oraz weryfikację implementacji pętli opóźniających na poziomie rejestrów roboczych.

Tabela 2 - Szczegółowy listing assemblerowy programu z zadania 3, sterującego Portem A.

Adres	Instrukcja	Argumenty	Szczegółowy opis techniczny
00000036	SER	R24	Ustawienie rejestru R24 na 0xFF.
00000037	OUT	0x1A, R24	Konfiguracja DDRA (0x1A) – Port A jako wyjście.
00000038	LDI	R24, 0x01	Ładowanie stanu początkowego (dioda D0).
00000039	OUT	0x1B, R24	Zapis do PORTA (0x1B) – zapalenie pierwszej diody.
0000003A	LDI	R18, 0x3F	Ładowanie licznika opóźnienia 1000ms (część 1).
0000003B	LDI	R20, 0x0D	Ładowanie licznika opóźnienia 1000ms (część 2).
0000003C	LDI	R24, 0x03	Ładowanie licznika opóźnienia 1000ms (część 3).
0000003D	SUBI	R18, 0x01	Dekrementacja licznika (odejmowanie 1).
0000003E	SBCI	R20, 0x00	Odejmowanie z przeniesieniem (obsługa 24-bitowa).
0000003F	SBCI	R24, 0x00	Odejmowanie z przeniesieniem (obsługa 24-bitowa).

00000040	BRNE	PC-0x03	Powrót do odejmowania, jeśli licznik > 0.
00000041	RJMP	PC+0x0001	Przeskok wyrównujący czas wykonania instrukcji.
00000042	NOP		Cykl jałowy (No Operation).
00000043	LDI	R18, 0x01	Inicjalizacja i = 1 dla pierwszej pętli.
00000044	LDI	R19, 0x00	Wyższy bajt licznika pętli i.
00000045	RJMP	PC+0x0017	Skok do sprawdzenia warunku pętli na adres 005C.
00000046	IN	R20, 0x1B	Odczyt aktualnej wartości z PORTA.
00000047	LDI	R24, 0x01	Przygotowanie maski do przesunięcia bitowego.
00000048	LDI	R25, 0x00	Wyższy bajt maski.
00000049	MOV	R0, R18	Kopiowanie i do R0 (licznik przesunięć).
0000004A	RJMP	PC+0x0003	Skok do sekcji przesuwania bitu.
0000004B	LSL	R24	Przesunięcie bitu w lewo (Logical Shift Left).
0000004C	ROL	R25	Przesunięcie przez flagę Carry (obsługa 16-bit).
0000004D	DEC	R0	Zmniejszenie licznika przesunięć.
0000004E	BRPL	PC-0x03	Powtarzaj przesuwanie, dopóki R0 jest dodatni.
0000004F	OR	R24, R20	Wykonanie operacji zapisu zaktualizowanego stanu do rejestru PORTA (zapalenie diody bez gaszenia pozostałych).
00000050	OUT	0x1B, R24	Zapisanie nowej wartości na fizyczny Port A.
00000051	LDI	R25, 0x3F	Opóźnienie 1000ms wewnątrz pętli (część 1).
00000052	LDI	R20, 0x0D	Opóźnienie 1000ms (część 2).
00000053	LDI	R24, 0x03	Opóźnienie 1000ms (część 3).
00000054	SUBI	R25, 0x01	Pętla odliczająca czas wewnątrz for.
00000055	SBCI	R20, 0x00	Odejmowanie z przeniesieniem.
00000056	SBCI	R24, 0x00	Odejmowanie z przeniesieniem.
00000057	BRNE	PC-0x03	Skok wstecz, dopóki delay nie minie.
00000058	RJMP	PC+0x0001	Synchronizacja.
00000059	NOP		Cykl jałowy.
0000005A	SUBI	R18, 0xFF	i++ (dodanie 1 przez odjęcie -1).
0000005B	SBCI	R19, 0xFF	Inkrementacja wyższego bajtu zmiennej i.
0000005C	CPI	R18, 0x08	Sprawdzenie czy i < 8.
0000005D	CPC	R19, R1	Porównanie wyższego bajtu z zerem (R1).
0000005E	BRLT	PC-0x18	Jeśli i < 8, wróć do zapalania kolejnej diody.
0000005F	LDI	R18, 0x07	Inicjalizacja i = 7 dla drugiej pętli.
00000060	LDI	R19, 0x00	Wyższy bajt dla i = 7.
00000061	RJMP	PC+0x0017	Skok do sprawdzenia warunku pętli na adres 00000078.
00000062	IN	R20, 0x1B	Odczyt stanu portu do wygaszania.
00000063	LDI	R24, 0x01	Przygotowanie maski do XOR.
00000064	LDI	R25, 0x00	Starszy bajt maski bitowej.
00000065	MOV	R0, R18	Kopiowanie licznika „i” do rejestru przesunięć R0.
00000066	RJMP	PC+0x0003	Skok do sekcji przesuwania maski bitowej
00000067	LSL	R24	Przesuwanie bitu maski o i pozycji w lewo.
00000068	ROL	R25	Przesunięcie starszego bajtu przez flagę Carry
00000069	DEC	R0	Zmniejszenie licznika pętli przesunięcia.
0000006A	BRPL	PC-0x03	Kontynuacja przesuwania maski, aż licznik R0 będzie mniejszy od 0
0000006B	EOR	R24, R20	XOR – zgaszenie bitu i w rejestrze.(wygaszenie diody w sekwencji powrotnej)
0000006C	OUT	0x1B, R24	Aktualizacja fizycznego stanu PORTA.
0000006D	LDI	R25, 0x9F	Opóźnienie 500ms (część 1).
0000006E	LDI	R20, 0x86	Opóźnienie 500ms (część 2).
0000006F	LDI	R24, 0x01	Opóźnienie 500ms (część 3).
00000070	SUBI	R25, 0x01	Pętla odliczająca 500ms.
00000071	SBCI	R20, 0x00	Obsługa flagi przeniesienia (Carry) w pętli opóźnienia 500ms.
00000072	SBCI	R24, 0x00	Obsługa flagi przeniesienia (Carry) w pętli opóźnienia 500ms.
00000073	BRNE	PC-0x03	Skok wstecz do odliczania, dopóki wynik będzie różny od zera
00000074	RJMP	PC+0x0001	Krótki skok w celu precyzyjnej synchronizacji czasu
00000075	NOP		Cykl jałowy (oczekiwanie 1 cyklu zegara)
00000076	SUBI	R18, 0x01	i-- (zmniejszenie licznika pętli).

00000077	SBC	R19, R1	Odejmij z przeniesieniem (R1=0).
00000078	CP	R1, R18	Sprawdzenie czy $0 < i$.
00000079	CPC	R1, R19	Porównanie starszego bajtu licznika z zerem (rejestr R1)
0000007A	BRLT	PC-0x18	Jeśli $i > 0$, wróć do gaszenia diody.
0000007B	RJMP	PC-0x0038	Skok na początek pętli głównej – while(1){}
0000007C	CLI		Globalne wyłączenie przerw.
0000007D	RJMP	PC-0x0000	Pętla nieskończona (zatrzymanie programu).

Analiza tabeli:

Kluczowym elementem sterowania przepływem programu są operacje warunkowe (np. adresy 00000040, 0000005E, 0000007A). Procesor realizuje je dwuetapowo: najpierw wykonuje porównanie wartości lub operację arytmetyczną, która ustawia odpowiednie flagi w rejestrze stanu (SREG), a następnie instrukcja skoku warunkowego (np. BRNE lub BRLT) decyduje o zmianie licznika programu (PC) na podstawie tych flag. Pozwala to na sprawną implementację pętli for oraz precyzyjne odmierzenie czasu poprzez wielokrotne powtarzanie sekwencji dekrementacji.

4. Wnioski

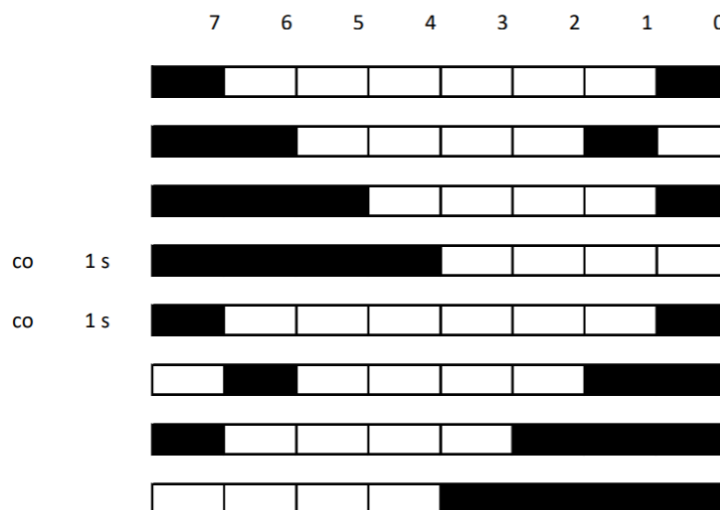
- **Złożoność operacji bitowych:** Analiza wykazała, że proste operacje w języku C, takie jak przesunięcie bitowe ($1 \ll i$), są dla procesora AVR operacjami kosztownymi. Architektura ta nie posiada instrukcji przesunięcia o zmienną liczbę pozycji, dlatego kompilator realizuje to zadanie poprzez iteracyjne wywoływanie instrukcji LSL i ROL wewnątrz pętli asemblerowej.
- **Optymalizacja rozmiaru i czasu:** Kompilator sprytnie optymalizuje kod, używając instrukcji SUBI R18, 0xFF zamiast klasycznego dodawania, co pozwala na jednoczesną obsługę flagi przeniesienia (Carry) przy operacjach na typach 16-bitowych. To pokazuje, że kod wynikowy jest zorientowany na maksymalną wydajność przy ograniczonych zasobach mikrokontrolera.
- **Realizacja opóźnień czasowych:** Funkcja `_delay_ms` nie jest magicznym poleceniem, lecz precyzyjnie wyliczoną pętlą „marnującą” cykle procesora. Liczba iteracji pętli opartej na instrukcjach SUBI, SBCI i BRNE jest ściśle powiązana z częstotliwością taktowania F_{CPU} (1 MHz), co zapewnia dokładność odmierzanego czasu (1s i 0.5s).
- **Logika sterowania wyjściami:** Wykorzystanie różnych operatorów logicznych pozwoliło na uzyskanie dwóch efektów: operator OR (suma logiczna) umożliwił kumulatywne zapalenie diod, natomiast operator XOR (EOR w asemblerze) pozwolił na selektywne wygaszanie bitów w drodze powrotnej sekwencji.
- **Stabilność pracy procesora:** Kod kończy się instrukcją wyłączenia przerw CLI oraz pętlą nieskończoną RJMP PC-0x0000, co jest dobrą praktyką inżynierską – zapobiega to nieprzewidywalnemu zachowaniu procesora po wykonaniu głównego algorytmu.

Zadanie 5

1. Opis ćwiczenia

Celem zadania było przygotowanie programu sterującego portem A mikrokontrolera, który realizuje złożoną animację świetlną polegającą na progresywnym przemieszczaniu się segmentów diod LED od skrajnych bitów rejestru ku jego środkowi. Program w pierwszej fazie manipuluje stanami logicznymi pinów PA7-PA0 w taki sposób, aby uzyskać efekt narastającego paska od lewej strony portu, przy jednoczesnych zmianach po prawej stronie, natomiast w drugiej części sekwencji następuje lustrzane odbicie animacji, w którym aktywność diod koncentruje się na przeciwległej krawędzi rejestru. Każdy krok operacji na bitach wyświetlany jest w stałym interwale wynoszącym 1 s, co pozwala na wyraźną obserwację zmian.

Zadanie 5



Rysunek 3 - graficzna interpretacja zadania nr5

2. Kod źródłowy programu

```
/*
 * Laboratorum1.c
 *
 * Created: 21.04.2026 16:26:01
 * Author : student_pl
 */

#define F_CPU 1000000UL
#include <avr/io.h>
#include <util/delay.h>

int main(void)
{
    DDRA = 0xFF;
    while (1)
    {
        PORTA = 0b10000001;
        _delay_ms(1000);
        for(int i=1; i<=3; i++){
            PORTA |= 1 << i;
            PORTA ^= 0b11000000;
```

```

        if(i==3){
            PORTA &= ~(1<<6 | 1<<7);
        }
        _delay_ms(1000);
    }

    PORTA = 0b10000001;
    _delay_ms(1000);

    for(int i=6; i>=4; i--){
        PORTA |= 1<<i;
        PORTA ^= 0b00000011;
        if(i==4){
            PORTA &= ~(1<<0 | 1<<1);
        }
        _delay_ms(1000);
    }
}

```

3. Analiza niskopoziomowa programu

Poniższe zestawienie szczegółowo obrazuje transformację złożonego algorytmu sterowania (wykorzystującego instrukcje warunkowe „if”) na niskopoziomowe skoki procesora AVR. Tabela dokumentuje unikalne podejście kompilatora do operacji XOR (instrukcja EOR), która służy do negacji stanu konkretnych diod, oraz operacji AND, wykorzystywanej do selektywnego wygaszania bitów. Analiza ta pozwala na weryfikację poprawności działania mechanizmów sterowania przepływem (BRNE, BRLT) w zależności od aktualnego stanu liczników pętli.

Tabela 3 - Szczegółowy listing assemblerowy programu z zadania 5, sterującego Portem A.

Adres	Instrukcja	Argumenty	Opis techniczny operacji
00000036	SER	R24	Ustawienie rejestru R24 na 0xFF.
00000037	OUT	0x1A, R24	Zapis 0xFF do DDRA (Port A jako wyjście).
00000038	LDI	R24, 0x81	Ładowanie maski 0x81 (PA0 i PA7).
00000039	OUT	0x1B, R24	Zapis do PORTA (zapalenie skrajnych diod).
0000003A	LDI	R18, 0x3F	Delay 1: Licznik bajt 1.
0000003B	LDI	R20, 0x0D	Delay 1: Licznik bajt 2.
0000003C	LDI	R24, 0x03	Delay 1: Licznik bajt 3.
0000003D	SUBI	R18, 0x01	Pętla opóźnienia: odejmowanie 1.
0000003E	SBCI	R20, 0x00	Odejmowanie z pożyczką (bajt 2).
0000003F	SBCI	R24, 0x00	Odejmowanie z pożyczką (bajt 3).
00000040	BRNE	PC-0x03	Powrót do 0x3D jeśli wynik != 0.
00000041	RJMP	PC+0x0001	Skok wyrównujący czas.
00000042	NOP		Pusta instrukcja (czekanie).
00000043	LDI	R18, 0x01	i = 1 (część młodsza).
00000044	LDI	R19, 0x00	i = 1 (część starsza).
00000045	RJMP	PC+0x0021	Skok do sprawdzenia warunku (0x66).
00000046	IN	R20, 0x1B	Odczyt stanu PORTA.
00000047	LDI	R24, 0x01	Maska do przesunięcia (1).
00000048	LDI	R25, 0x00	Starszy bajt maski.
00000049	MOV	R0, R18	Kopiowanie i do licznika przesunięć.
0000004A	RJMP	PC+0x0003	Skok do przesunięć.
0000004B	LSL	R24	Przesunięcie bitu w lewo.
0000004C	ROL	R25	Przesunięcie przez Carry (16-bit).
0000004D	DEC	R0	Zmniejszenie licznika pętli przesunięć.
0000004E	BRPL	PC-0x03	Powrót do 0x4B jeśli licznik >= 0.

0000004F	OR	R24, R20	`PORTA
00000050	OUT	0x1B, R24	Fizyczny zapis na piny portu.
00000051	IN	R25, 0x1B	Odczyt stanu portu do XOR.
00000052	LDI	R24, 0xC0	Maska 0xC0 (bity 6 i 7).
00000053	EOR	R24, R25	Przełączenie stanu PA6 i PA7.
00000054	OUT	0x1B, R24	Fizyczna aktualizacja portu.
00000055	CPI	R18, 0x03	Porównanie i z 3.
00000056	CPC	R19, R1	Porównanie starszego bajtu i.
00000057	BRNE	PC+0x04	Skok nad if jeśli i != 3.
00000058	IN	R24, 0x1B	Odczyt portu do gaszenia.
00000059	ANDI	R24, 0x3F	Wygaszenie PA6 i PA7 (maska 0x3F).
0000005A	OUT	0x1B, R24	Fizyczne wygaszenie diod.
0000005B	LDI	R25, 0x3F	Delay 2: Licznik bajt 1.
0000005C	LDI	R20, 0x0D	Delay 2: Licznik bajt 2.
0000005D	LDI	R24, 0x03	Delay 2: Licznik bajt 3.
0000005E	SUBI	R25, 0x01	Pętla opóźnienia: odejmowanie 1.
0000005F	SBCI	R20, 0x00	Odejmowanie z pożyczką.
00000060	SBCI	R24, 0x00	Odejmowanie z pożyczką.
00000061	BRNE	PC-0x03	Powrót do 0x5E jeśli != 0.
00000062	RJMP	PC+0x0001	Skok wyrównujący czas.
00000063	NOP		Pusta instrukcja.
00000064	SUBI	R18, 0xFF	Inkrementacja i (dolny bajt).
00000065	SBCI	R19, 0xFF	Inkrementacja i (górny bajt).
00000066	CPI	R18, 0x04	Czy i < 4?
00000067	CPC	R19, R1	Dopełnienie porównania 16-bitowego.
00000068	BRLT	PC-0x22	Skok na początek pętli (0x46).
00000069	LDI	R24, 0x81	Przywrócenie stanu 0x81.
0000006A	OUT	0x1B, R24	Zapis na PORTA.
0000006B	LDI	R25, 0x3F	Delay 3: Licznik bajt 1.
0000006C	LDI	R18, 0x0D	Delay 3: Licznik bajt 2.
0000006D	LDI	R20, 0x03	Delay 3: Licznik bajt 3.
0000006E	SUBI	R25, 0x01	Pętla opóźnienia: odejmowanie 1.
0000006F	SBCI	R18, 0x00	Odejmowanie z pożyczką.
00000070	SBCI	R20, 0x00	Odejmowanie z pożyczką.
00000071	BRNE	PC-0x03	Powrót do 0x6E jeśli różne od zera.
00000072	RJMP	PC+0x0001	Skok wyrównujący.
00000073	NOP		Pusta instrukcja.
00000074	LDI	R18, 0x06	i = 6 (druga pętla).
00000075	LDI	R19, 0x00	i = 6 (górny bajt).
00000076	RJMP	PC+0x0021	Skok do warunku (0x97).
00000077	IN	R20, 0x1B	Odczyt PORTA.
00000078	LDI	R24, 0x01	Maska do przesunięcia.
00000079	LDI	R25, 0x00	Starszy bajt maski.
0000007A	MOV	R0, R18	Licznik przesunięć = i.
0000007B	RJMP	PC+0x0003	Skok do przesunięć.
0000007C	LSL	R24	Przesunięcie bitu w lewo.
0000007D	ROL	R25	Przesunięcie przez Carry.
0000007E	DEC	R0	Zmniejszenie licznika.
0000007F	BRPL	PC-0x03	Powtórz przesunięcie.
00000080	OR	R24, R20	`PORTA
00000081	OUT	0x1B, R24	Aktualizacja pinów.
00000082	IN	R25, 0x1B	Odczyt portu do XOR.
00000083	LDI	R24, 0x03	Maska 0x03 (piny PA0 i PA1).
00000084	EOR	R24, R25	Przełączenie PA0 i PA1.
00000085	OUT	0x1B, R24	Aktualizacja portu.
00000086	CPI	R18, 0x04	Czy i == 4?

00000087	CPC	R19, R1	Porównanie starszego bajtu i.
00000088	BRNE	PC+0x04	Skok nad if jeśli nie.
00000089	IN	R24, 0x1B	Odczyt portu do gaszenia.
0000008A	ANDI	R24, 0xFC	Wygaszenie PA0 i PA1 (maska 0xFC).
0000008B	OUT	0x1B, R24	Fizyczne wygaszenie.
0000008C	LDI	R24, 0x3F	Delay 4: Licznik bajt 1.
0000008D	LDI	R25, 0x0D	Delay 4: Licznik bajt 2.
0000008E	LDI	R20, 0x03	Delay 4: Licznik bajt 3.
0000008F	SUBI	R24, 0x01	Pętla opóźnienia: odejmowanie 1.
00000090	SBCI	R25, 0x00	Odejmowanie z pożyczką.
00000091	SBCI	R20, 0x00	Odejmowanie z pożyczką.
00000092	BRNE	PC-0x03	Powrót do 0x8F jeśli != 0.
00000093	RJMP	PC+0x0001	Skok wyrównujący.
00000094	NOP		Pusta instrukcja.
00000095	SUBI	R18, 0x01	Dekrementacja i (R18 - 1).
00000096	SBC	R19, R1	Dekrementacja i (starszy bajt).
00000097	CPI	R18, 0x04	Czy i >= 4?
00000098	CPC	R19, R1	Dopełnienie porównania 16-bitowego.
00000099	BRGE	PC-0x22	Skok na początek pętli (0x77).
0000009A	RJMP	PC-0x0062	Główna pętla while(1) – skok do 0x38.
0000009B	CLI		Globalne wyłączenie przerwań.
0000009C	RJMP	PC-0x0000	Pułapka (skok do samego siebie).

Analiza tabeli:

Analiza niskopoziomowa ujawnia rozbudowaną strukturę decyzyjną programu. Kluczowym elementem są sekcje warunkowe (np. adresy 0055–0057 oraz 0086–0088), gdzie instrukcje CPI (Compare with Immediate) i CPC (Compare with Carry) weryfikują stan licznika pętli i. Na ich podstawie instrukcja BRNE decyduje o wykonaniu dodatkowych operacji modyfikacji portu (instrukcje ANDI pod adresem 0059 oraz 008A). Pokazuje to mechanizm "bramkowania" kodu, gdzie wybrane fragmenty algorytmu są aktywowane tylko dla specyficznych iteracji pętli.

4. Wnioski:

- **Zastosowanie logiki bitowej do selektywnego gaszenia:** W przeciwieństwie do zadania 3, tutaj wykorzystano operator AND z maską (np. 0x3F lub 0xFC), co pozwala na wymuszone zerowanie konkretnych bitów (PA6-PA7 lub PA0-PA1) bez wpływu na pozostałe diody.
- **Koszt instrukcji warunkowych:** Każda instrukcja „if” w kodzie C generuje dodatkowy narzut w assemblerze w postaci sekwencji porównań i skoków. Analiza wykazała, że obsługa warunków dla zmiennych 16-bitowych wymaga użycia pary instrukcji CPI/CPC, co jest niezbędne dla poprawnego badania stanu rejestru flag SREG.
- **Manipulacja stanem przez EOR:** Zastosowanie instrukcji EOR (XOR) pod adresami 0053 i 0084 pozwoliło na uzyskanie efektu "migania" lub przełączania stanu diod, co demonstruje przewagę operacji bitowych nad wielokrotnym wpisywaniem stałych wartości do rejestru PORTA.
- **Determinizm czasowy:** Mimo różnej ścieżki wykonania kodu (zależnie od tego, czy warunek if jest spełniony), różnice w liczbie cykli zegarowych są całkowicie maskowane przez pętle opóźnienia oparte na SBCI i BRNE, co zapewnia wizualną płynność animacji niezależnie od złożoności obliczeń.
- **Bezpieczeństwo końcowe:** Zastosowanie sekwencji CLI oraz pułapki RJMP PC gwarantuje, że po zakończeniu skomplikowanej sekwencji sterującej mikrokontroler nie zacznie wykonywać przypadkowych instrukcji z pustych obszarów pamięci.

Wnioski końcowe:

Działania sekwencyjne w mikrokontrolerach wymagają nie tylko poprawnej logiki w języku programowania, ale także świadomości ograniczeń sprzętowych. Zastosowanie pętli for oraz instrukcji warunkowych pozwala na tworzenie złożonych animacji świetlnych, jednak to optymalizacja na poziomie assemblera decyduje o finalnej wydajności i rozmiarze kodu. Laboratoryjne zadania potwierdziły, że stabilność pracy układu zapewnia odpowiednia struktura kodu, kończąca się nieskończoną pętlą, co zapobiega niekontrolowanemu wykonywaniu instrukcji po zakończeniu głównego algorytmu.